Analisi estesa: propagazione dei flag e metadati affini

Oltre ai soli TypeFlags, BabyC tiene **più strati paralleli** di "verità di tipo". Quando non sono allineati nascono i bug tipo *f() (sempre 4 byte) o le divergenze soft EXEC=both.

---

1. Inventario completo sul nodo IrExpr

Campo	Ruolo	Chi lo scrive	Chi lo legge
TypeFlags	classificazione grezza (INT/FP/PTR/UNSIGNED/…)	IrAnnotateExpr	eval, ops, lower gate, fuse
ValWidth	IR_VW_I8/16/32/64	annotate da Sizeof/I32	narrow, wide, shift
ElemSize	scala puntatore / load element	compile+annotate	IrPtrElemSizeOf, p±n, Index
I32	BaseType target (cast, const tipizzata)	compile cast/const	cast value, unsigned
ConstVal	letterale **oppure** size pointee su TokenCast	compile	* (T*)p, fold
BitOffset/Width/Signed	bitfield	resolve member	load/store bitfield
SignedFast	cache "sicuro signed"	IrApplyBinOp (lazy)	solo binop
WideCache	cache wide (0/1/2)	IrApplyBinOp se unsigned	solo binop
CachedFunc/Val	bind nome	compile/runtime	call, global
LocalSlot/ParamSlot	legame frame	compile	load/store slot
SourceLine/Pos	diagnostica	compile	ProgramFail caret

SignedFast / WideCache non sono fonti di verità: sono memo di TypeFlags + walk. Se annotate è sbagliato, la cache **congela** l'errore (veloce ma coerente con il bug).

---

2. Inventario parallelo sul frame (IrCallFrame)

Campo	Equivalente nodo	Note
LocalTyp[]	tipo dello slot	"type of truth" B.1
LocalFromType[]	pointee / struct dietro ptr	peel p->m
LocalIsPtr / LocalIsFptr / LocalElemIsPtr	≈ IR_TF_PTR	path load

Campo	Equivalente nodo	Note
LocalElemSize / LocalPointeeSize	≈ ElemSize	**semantica diversa** (spesso sizeof(void*) vs pointee)
LocalInit / LocalInitMap	≈ IR_TF_DEFINITELY_INIT	SAFE; map per byte
LocalArraySize / LocalDims / NDims	layout array	OOB, stride
ParamTyp / ParamIsPtr / ParamVal	formals	call pack
Decl* su IrStmt	blueprint a compile	copiato nel frame a decl-exec

Regola empirica:
albero = verità per *espressione*; frame = verità per *storage*.
Il lineare/VM quasi non legge TypeFlags: usa slot + IrNarrowToLocalType + Imm stampato al lower.

---

3. Matrice di propagazione per *kind*

text

Const —annotate—► INT|FP|PTR + ValWidth + (UNSIGNED se I32 unsigned)

Ident —slot/typ—► TypeFlags + ElemSize da LocalTyp/ParamTyp/global

BinOp —OR figli—► UNSIGNED da Left (shift/div); PTR se ± su ptr; INT altrimenti

Assign —target——► non "è" un valore; side-effect su frame init

Call —compile—► spesso IR_TF_PTR|RESOLVED (designator); ReturnType NON sul nodo

Unary * —Left——► load: size da ResolvePtr(Left), non da TypeFlags del *

Unary & —Left——► PTR; ElemSize del pointee

Unary Cast ————► ConstVal=size (ptr) o 0+I32 (value); UNSIGNED/FP da I32

Index —Left————► ElemSize da array/ptr Left; ValWidth da elemento

Member —struct—► bitfield + ValWidth membro; PTR se membro array decay

Ternary ————————► union conservativa dei due bracci (spesso debole)

Casi già noti / trattati

- IR_TF_* → eval / lower gate / IR_LIN_F_UNSIGNED
- LocalInit → SAFE tree + ora LoadLocal linear
- Call IR_TF_PTR che bloccava il lower (escluso dal gate)

Casi simili non trattati prima (estensione)

Situazione	Cosa manca sul nodo	Effetto
IrExCall return pointee	niente ElemSize/ValWidth del *pointee*	*f() leggeva sempre 4 B → **fix W11** via ReturnType in IrResolvePtr

Situazione	Cosa manca sul nodo	Effetto
f()[i] / *(f()+i)	scale da IrPtrElemSizeOf(Call) = 0 (no Parser)	scale può restare sizeof(int) se non passa da ResolvePtr annotato
(*f()).m / f()->m	ST da ReturnType (già best-effort in member)	OK se Ident risolto; debole se solo Left Ident
Cast value (unsigned)x	ConstVal=0, I32=target	ops usa I32/TypeFlags; linear Bin vede UNSIGNED solo se annotate ha stampato il bit sul **BinOp padre**
Cast ptr (char*)p	ConstVal=1 (size), non ValWidth del load *	* sul cast usa ConstVal walk — path dedicato, non TypeFlags
Bitfield member	Bit* su expr Member	linear LoadIndex Tok=member: dipende da IrSafeMemberAddr, non da Bit* sul reg
Array decay a in a+i	PTR + ElemSize elemento	se ElemSize manca → scale 4 default
T *a[N]	LocalElemIsPtr + ElemSize word	confusione ElemSize vs PointeeSize (documentata in ResolvePtr)
Struct-by-value local	no IR_TF_*; owned blob	init map byte; scalar rvalue member non marca parent (fix 255)
Fptr / table	PTR + Fptr flag frame	call through: CachedFunc spesso assente
Ternary tipizzato misto	annotate debole	un braccio FP e uno INT: coerzione a runtime, flag nodo ambigui
Comma operator	flag del Right	Left side-effect only
Compound s.m +=	UNSIGNED del **membro**	linear stamp da Left Ident struct, non dal member type → possibile soft mismatch
Kernel / ForFp	ForFpKind su stmt	path ir_fast ha **altra** logica size/unsigned (storicamente bug uchar in kernel)
Static local	tabella persist, non TypeFlags	init tra chiamate; flag nodo per-ident freschi ogni body

---

4. Catene di propagazione “simili” al buco Call

Stesso schema del bug W6/W11:

text

operazione su valore puntatore

→ serve larghezza/signedness del *pointee*

→ il nodo in mano è Call / Cast / Index / Member / Unary*

→ se si usa un default (int/4) invece della catena ReturnType/FromType/ConstVal

→ lettura o scala sbagliata

Pattern	Fonte corretta	Default sbagliato tipico
*f()	ReturnType->FromType	int/4
f()[0]	stesso via ResolvePtr(Left=Call)	int/4
*(p+i)	IrPtrElemSizeOf(p) o LocalPointeeSize	sizeof(int)
(T)p	cast ConstVal + I32	int se ConstVal perso
*p Ident	LocalFromType / ParamTyp	int
s.m bitfield	BitWidth su Member	full int load
a[i] uchar	ElemBase unsigned	sign-extend (bug kernel storico)

---

5. Propagazione Decl → frame → nodo

text

IrStDecl (DeclIsPtr, DeclFromType, DeclElemSize, DeclPointeeSize, DeclArraySize, …)

| exec

▼

IrCallFrame Local*[]

| annotate Ident (LocalSlot)

▼

IrExpr TypeFlags | ElemSize | ValWidth

Rotture tipiche

1. Decl copia DeclElemSize = sizeof(void*) per array di ptr, ma annotate usa quello come scala di p+1 su elemento sbagliato.
2. LocalPointeeSize pieno (B.2) vs clamp storico 255 — path diversi se un ramo legge ancora ElemSize.
3. Multi-decl T *p, *q — un solo DeclFromType deve applicarsi a entrambi gli slot.
4. Typedef S* — DeclFromType deve essere S, non il typedef opaco senza Sizeof.

---

6. Linear / VM: secondo canale di flag

Meccanismo	Cosa propaga
IR_LIN_F_UNSIGNED / WIDE in Imm	TypeFlags al **lower-time** sul BinOp

Meccanismo	Cosa propaga
IrNarrowToLocalType su StoreLocal	larghezza/segno da **frame**, non dal nodo RHS
IdxTab[] / GlobTab[]	expr originali per LoadIndex (member/index)
Nessun RegFlags[]	dopo un load, il registro non ricorda UNSIGNED/ValWidth

Situazioni simili ancora aperte

- Dopo LoadLocal di unsigned char, un >> successivo nel **stesso** linear block usa UNSIGNED solo se il **BinOp** annotato lo aveva; non perché il reg è "uchar".
- IrLinCall non restituisce metadati di return type al Dest reg → *-like uso del risultato call nel linear è raro (call nested è tree).

---

7. Fast-path / kernel (ForFpKind, ir_fast)

Terzo canale, indipendente da TypeFlags e da IR_LIN_*:

- Decide su forma dello stmt (IR_FP_KERNEL_FOR, PTR_WALK, …).
- Usa spesso size fissi o LocalElemSize senza ripassare da annotate.
- **Stesso anti-pattern**: default signed int su elementi stretti → checksum uchar sbagliato (repro storici).

Qualsiasi fix solo su IrAnnotateExpr **non** basta se il kernel non consulta i flag.

---

8. Flag di controllo (env) e interazione

Già visti; estensione:

Flag	Interazione con TypeFlags
BABYC_SAFE / NO_UNINIT	LocalInit vs IR_TF_DEFINITELY_INIT
BABYC_NO_IR	nessun annotate; solo token Value.Typ
BABYC_EXEC=vm	solo Imm + frame; TypeFlags solo al lower
BABYC_NO_FUSE	ignora IR_TF_FUSABLE
LinExecDepth	nested call: tree puro → TypeFlags di nuovo vivi

---

9. Mappa "fonte di verità" per operazione

Operazione	Fonte preferita	Fallback pericoloso
/ % >> signedness	TypeFlags / SignedFast	walk lento; se cache stale OK

Operazione	Fonte preferita	Fallback pericoloso
load *p	ResolvePtr → FromType / cast ConstVal	**int/4**
load *f()	ReturnType FromType	**int/4** (chiuso)
p+n scale	ElemSize / PointeeSize	sizeof(int)
store narrow	LocalTyp slot	tronco a int
SAFE uninit	LocalInit(+map)	bit DEFINITELY_INIT solo tree
bitfield	Bit* su Member	full word
FP	IR_TF_FP / I32 TypeFP	bits in long long

---

10. Situazioni analoghe da aggredire (priorità)

1. **IrPtrElemSizeOf(Call) senza Parser** — scale f()+i ancora debole rispetto a ResolvePtr.
2. **Compound member UNSIGNED** — linear Bin da Ident struct, non dal tipo membro.
3. **Propagazione Ternary** — union dei bracci esplicita (FP vs INT).
4. **Kernel uchar/ushort** — allineare a IrLoadPtrElem unsigned (se ancora gap su path fast).
5. **RegFlags opzionali nel linear** — dopo LoadLocal, copiare UNSIGNED/ValWidth sullo slot reg per Bin successivi.
6. **Annotate Call return** — stampare sul nodo Call ElemSize/ValWidth del pointee a compile-time (evita lookup a ogni *f()).
7. **Caret su prod** — propagare SourceLine dell'expr nel Step (segnali errore, punto residuo 1–2).

---

11. Sintesi

La "propagazione dei flag" non è un solo bitfield: è **tre canali** (nodo, frame, fast-path) più **memo** (SignedFast/WideCache) e **Imm linear**.

Il pattern di guasto ricorrente è:

operazione che ha bisogno del pointee/elemento stretto, ma il nodo corrente non porta ElemSize/ValWidth/UNSIGNED e il codice usa un default a int.

TypeFlags copre bene classificazione grezza e gate; **non** sostituisce ReturnType, ConstVal dei cast, Bitfield, né i metadati frame. I fix robusti o (a) annotano il nodo al compile, o (b) risolvono il tipo completo al punto d'uso (IrResolvePtr / IrCallReturnType), mai un default silenzioso a 4 byte.